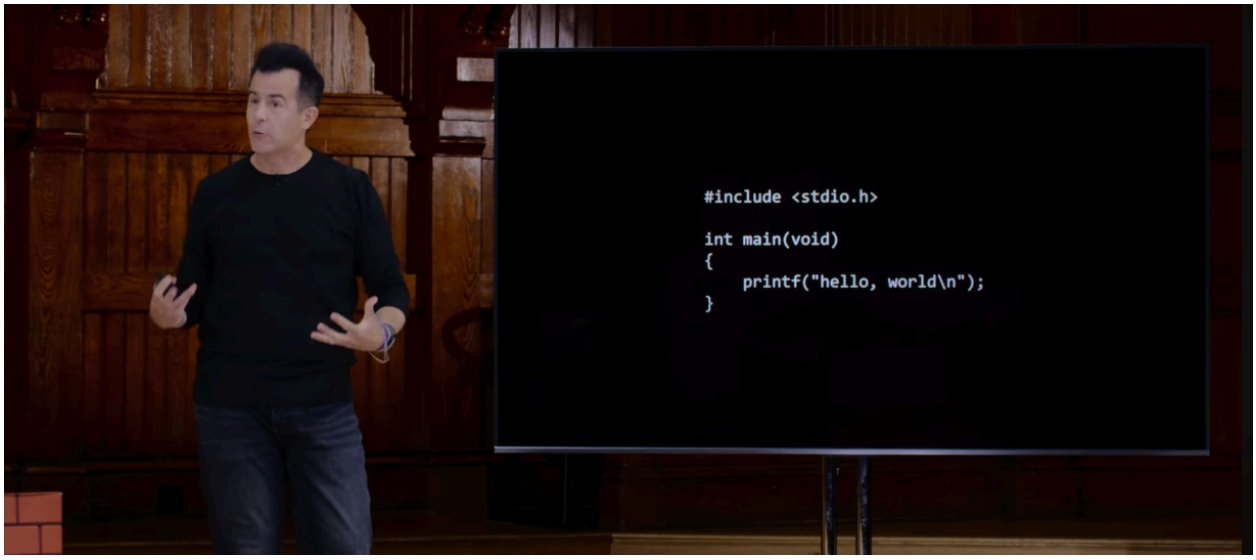
# CS50x 2026 - Lecture 1 Notes - C

By Wisdom

## Introduction




Programmers write Source codes and computers understand Machine codes (0s and 1s). C is a low-level computer program that is usually used for writing compilers, robots, and computer architectures.

I have always wanted to learn the C programming language, and this course by Harvard is a really good step in achieving this, as I have been able to gather resources to aid me in my C learning journey.

## From Scratch to C, Header Files

`, printf` is used to display results from a program. The **f** in `printf` means **format**, which allows you to control how text and values are displayed.

A **function** in code is similar to a function in basic mathematics, where you take an input like **x** and produce an output. However, functions in programming are more flexible because they can return different types of results, such as integers, booleans, characters, and more.

In C, `\n` at the end of a line means **start a new line**. This is part of a group called **escape sequences**, which are special characters used to represent actions or formatting inside text.

## Common Escape Sequences in C

- `\n` → New line
- `\t` → Horizontal tab (adds spacing like pressing the Tab key)
- `\\` → Prints a backslash
- `\"` → Prints a double quotation mark
- `\'` → Prints a single quotation mark
- `\r` → Carriage return (moves cursor to start of line)
- `\b` → Backspace (moves cursor one space backward)
- `\a` → Alert or beep sound

Most programming languages are **line-based** and are often thought to be interpreted line by line. This is somewhat of a **white lie**, but it is generally a helpful way to understand how programs appear to run, especially for beginners.

**Header files** are files that end in `.h`. They contain code written by other developers that anyone using the C language can include in their programs. Header files give access to what we call **libraries**, which provide prewritten functionality.

For example, `printf` is a feature of C, but to use it, you must include the library that contains it. This is done by including the header file **`stdio.h`**, which stands for **standard input/output**.

# Hello, You

hello.c

```c
#include <stdio.h>

int main(void)
{
    char answer[100];

    printf("What's your name? ");
    fgets(answer, sizeof(answer), stdin);

    printf("hello, %s", answer);
}

```

Unlike higher-level languages such as Python or JavaScript, C does not have many built-in abstractions, which means programmers must explicitly manage how data is stored and handled. One important concept I learned is that C does not have a built-in string data type. Instead, strings in C are represented as arrays of characters that end with a special character called the null terminator (`\0`). This null terminator signals to the program where the string ends. For example, the word “Wisdom” is stored in memory as individual characters followed by `\0`, which helps functions like `printf` know when to stop reading the string.

I also learned how memory allocation works when storing strings. When you write `char answer[100];`, you are telling the program to reserve space for 100 characters in memory. Each character takes one byte, so this allocates 100 bytes total. This size includes space for the characters the user types and the null terminator that marks the end of the string. The reason developers often reserve slightly more memory than they expect to use is to prevent overflow and allow the program to safely store user input without exceeding the allocated memory.

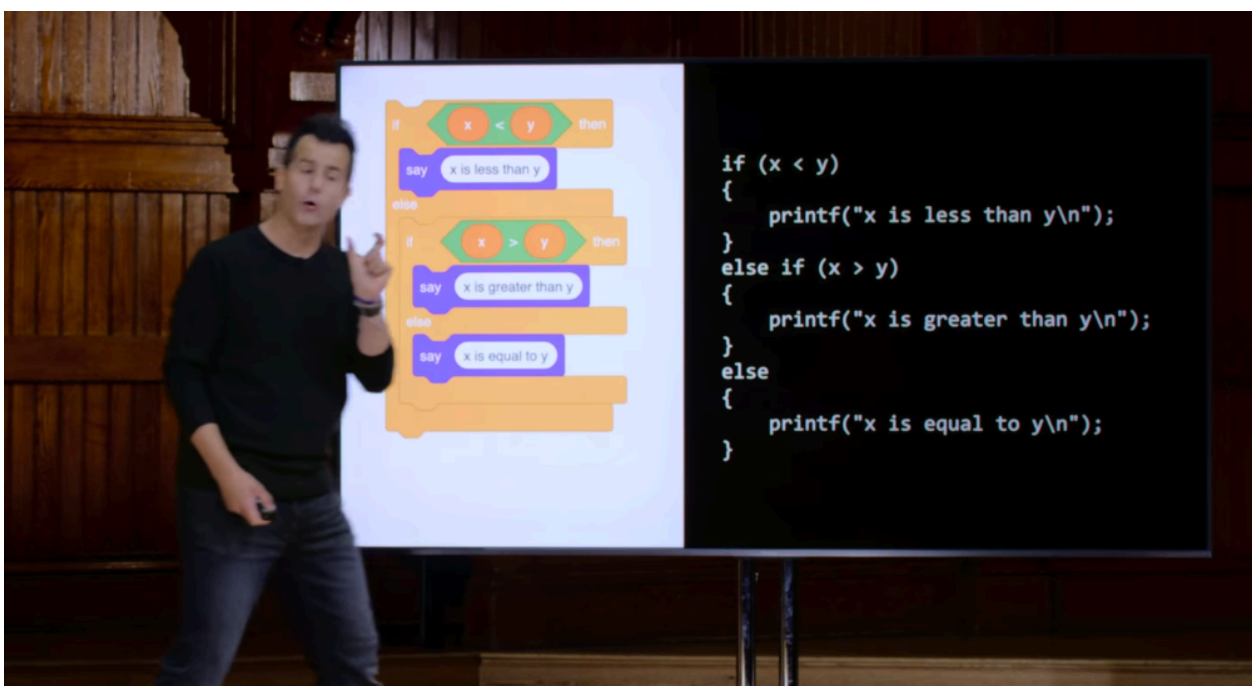
Another key concept I explored is how arrays work in C. Arrays store values sequentially in memory, meaning each character of a string is placed in consecutive memory locations. For example, if a user types “Hi,” the program stores ‘H’, ‘i’, and then `\0` in three adjacent memory spaces. Understanding this layout is important because many string operations in C rely on manually traversing memory until the null terminator is found.

I learned about the `scanf` function, which reads formatted input from the keyboard. When using `scanf("%s", answer);`, the `%s` format specifier tells the program to read a sequence of characters and store them in the `answer` array. However, `%s` has an important limitation: it stops reading input when it encounters whitespace such as a space, tab, or newline. This means if a user enters multiple words like a full name, `scanf("%s")` will only capture the first word. To reduce security risks, you can use `%99s` instead of `%s`, which limits the number of characters read and helps prevent buffer overflow. Buffer overflow occurs when input exceeds allocated memory, which can crash programs or introduce vulnerabilities.

I learned that `scanf("%s")` is often not ideal for reading full sentences or names containing spaces. To address this limitation, I found `fgets`, which is a safer and more flexible way to read user input. The `fgets` function reads an entire line of input, including spaces, and ensures the input does not exceed the buffer size. For example, `fgets(answer, sizeof(answer), stdin);` reads input safely while preventing memory overflow. One side effect of `fgets` is that it stores the newline character when the user presses Enter, which sometimes needs to be removed later in the program.

Overall, the most important mental model to keep in mind is that C treats strings as raw memory arrays of characters terminated by `\0`. Because C gives programmers full control over memory, it also requires careful attention to buffer sizes, input limits, and safe memory handling. Mastering these concepts forms the foundation for writing reliable, efficient, and secure C programs.

# Conditionals, Types & Loops

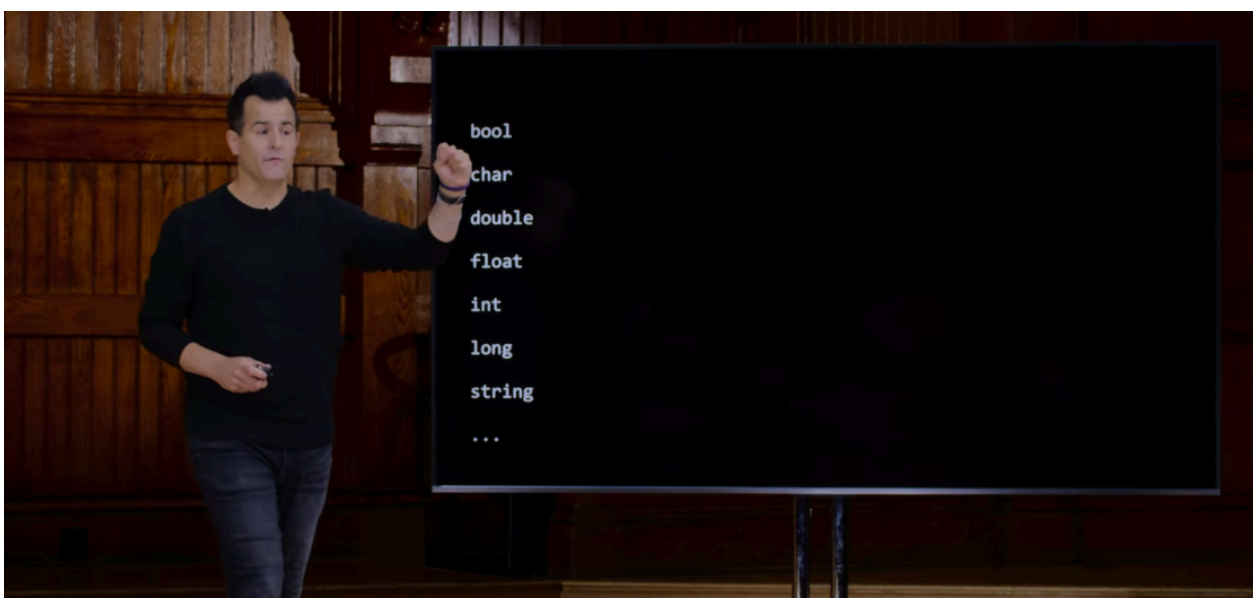


**Conditionals** in C are used to make decisions in a program based on whether certain conditions are true or false. They work similarly to conditionals in most other programming languages, using statements like `if`, `else if`, and `else` to control the flow of a program. Because of my previous background in engineering and programming, I was already familiar with the general concept; however, it has still been valuable revisiting it in C, especially understanding how strictly C evaluates conditions and handles logical operations.

Through this, I also reinforced how `else` statements allow programs to define alternative paths when a condition is not met, making programs more flexible and responsive to different inputs. Conditionals are important because they allow programs to behave differently depending on user input, data values, or system states, which is a core part of problem-solving in computer science.

**Fun fact:** In C, defining a variable is similar to languages like Rust because you must explicitly state the **type** of data the variable will hold before using it. For example, `int` stores whole numbers, `float` stores decimal numbers, and `char` stores single characters. This is known as **static typing**, and it helps the computer allocate memory efficiently while reducing unexpected errors. It also forces programmers to think more carefully about the kind of data they are working with, which can make programs more reliable and performant.

The equality sign in C is the "==" to compare two values in a conditional. C has a bunch of operators that help developers write more functional code.




The data types in C involve the following;

1. Bool for boolean, which is either true or false
2. Char, which is for an individual character
3. A float is a number that has a decimal, which uses 32 bits total to represent it. Double uses 64bit

4. Int is for integer, negative and positive, and the highest it can count is 4 billion, and uses 64 bits.

However, to print out these values from the computer, we use the following.

1. %s is for printing out a string
2. %c is for printing out characters
3. %f is for printing out a float
4. %i is for printing out an integer
5. %li for printing out long integer

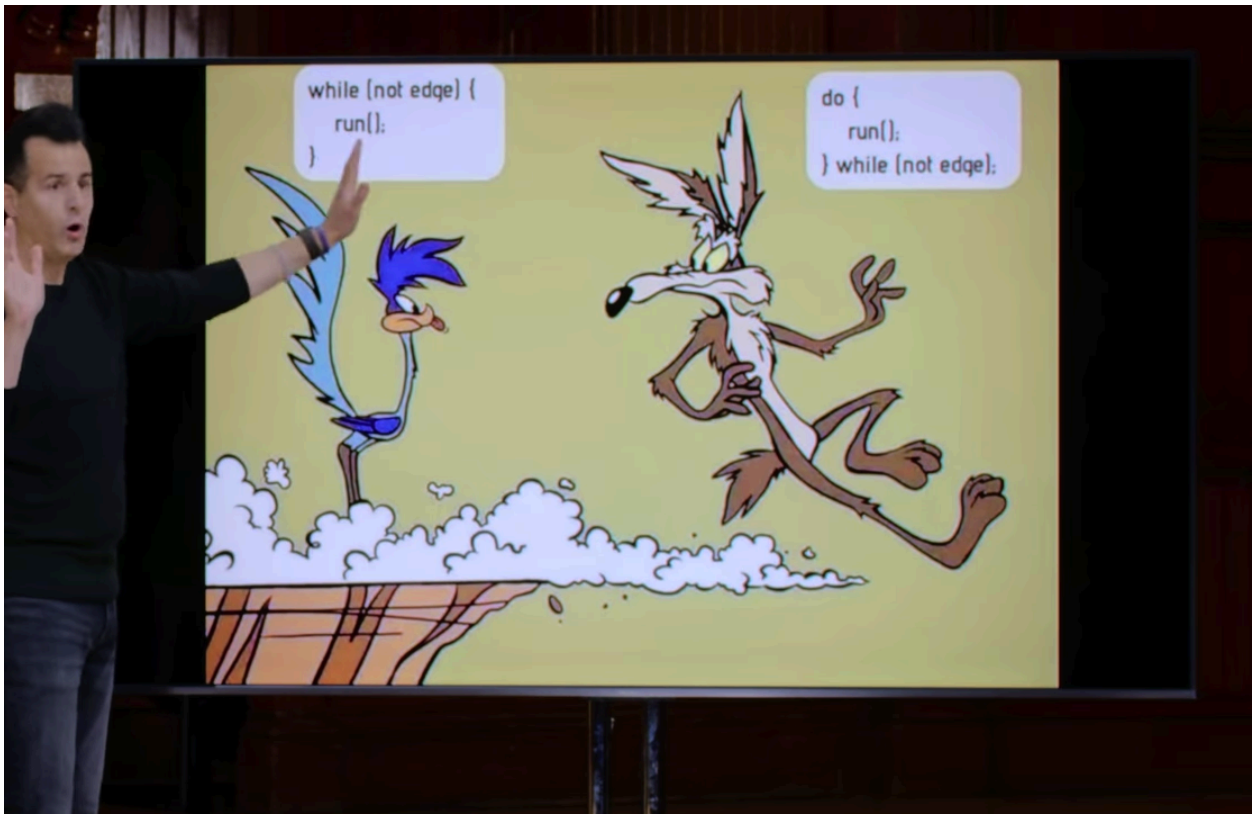
To understand these concepts in C, I built a simple calculator in C, which is capable of doing only simple maths calculations between two digits: Addition, Subtraction, Division, and Multiplication.

Check it out on my GitHub: Link.

To make things forever in C, simply do while (true) or while (1). This is kind of effective tho, but I'm still learning.

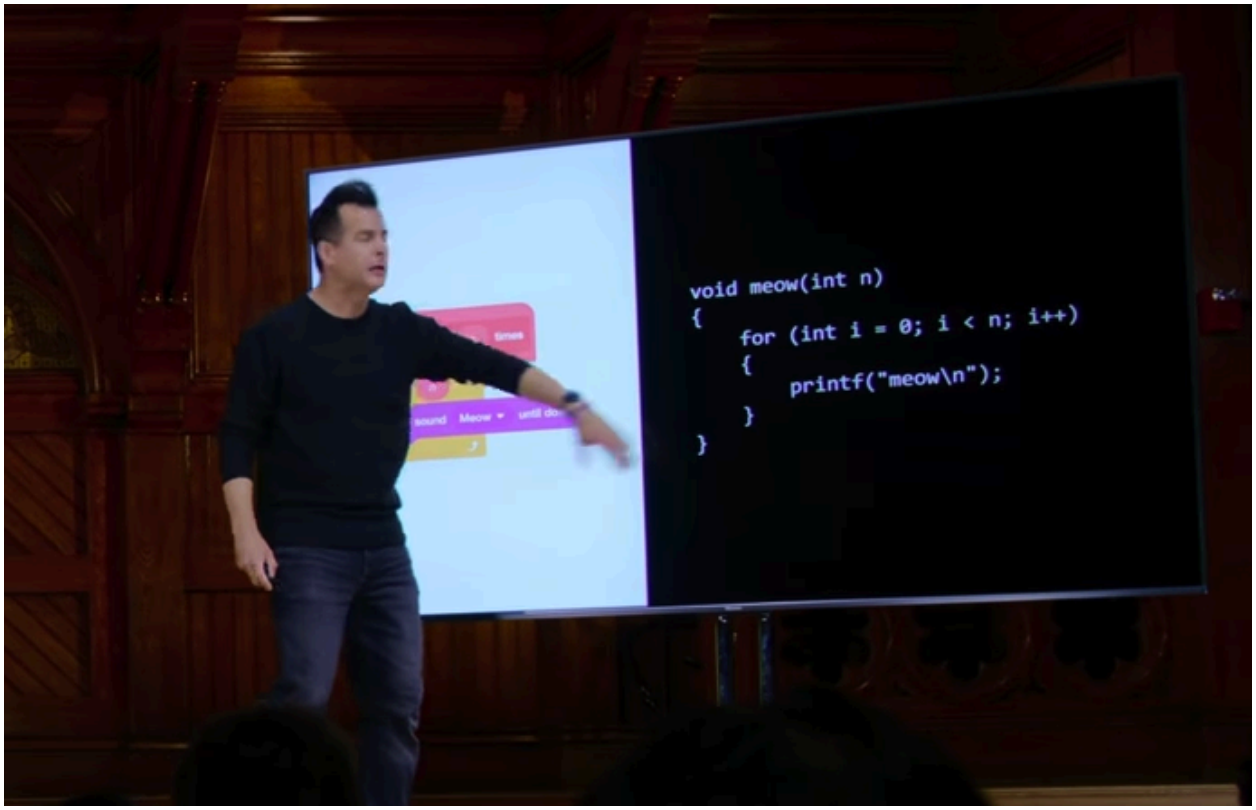
Continue in C means go back to the start of the loop and while break means to breakout of the loop and go to the next block of code in the program being written.

Do… while loop has its conditions at the bottom while a normal while loop happens at the head.




# Functions

A function in C is declared with a return type, a name, and a parenthesized list of parameters. Using the keyword `void` as the return type means the function does not return a value; using `void` inside the parentheses (`(void)`) means the function takes no arguments. Anything a function does besides returning a value, for example, printing to the screen, modifying a global variable, or writing to a file, is called a *side effect*. When a function accepts inputs, you list each parameter's type and name inside the parentheses; if it should return a value, use the appropriate return type instead of `void`.




Started coding, haha, stopped writing, and wrote more on VS Code.

The final project was to create a simple CLI calculator in C, and here it is.

Link: [https://github.com/owizdom/calc.c](https://github.com/owizdom/calc.c)